Introduction to Computing and LaTeX

Bash for the terminal, R for the data, LaTeX for the write-up.

Get the code

```
○○○ terminal
```

```
git clone https://github.com/seknewna/Introduction-to-Computing-and-LaTeX.git
cd Introduction-to-Computing-and-LaTeX
```

```
bash-scripting/scripts/*.sh
r-programming/chapters/*.Rmd
latex/index.html
```

Every script and chapter below is also shown in full on this page — clone only if you want to run them locally.

01

Bash Scripting

18 chapters, from your first terminal command to trap signals and named pipes. Course content adapted from Dave Eddy's free "Complete Bash Scripting Course" (ysap.sh), MIT-licensed source code.

00 Introduction

Every script starts with a shebang line telling the system which interpreter to use. This chapter sets expectations for the course and gets your environment ready before you touch a single line of real scripting.

▼ View script

○○○ bash-scripting/scripts/00-introduction.sh

```
#!/usr/bin/env bash
#
# Chapter 00 — Introduction
# Real commands referenced in this lesson of the course.

echo "This course lives entirely in the terminal — no separate app needed."
echo "Every later script in this repo starts with the same shebang line:"
echo '  #!/usr/bin/env bash'
```

01 Terminal & File Basics

Before scripting comes fluency in the terminal itself: moving around the filesystem, creating and removing files, revealing hidden dotfiles, searching inside files with grep, and paging through output that's too long for one screen.

▼ View script

○○○ bash-scripting/scripts/01-terminal-and-file-basics.sh

```
#!/usr/bin/env bash
#
# Chapter 01 — Terminal & File Basics
# Real commands from this lesson: cd, mkdir, touch, cat, echo, rm, pwd,
# cp, alias, man, ls -a, ls -l, grep -A/-B/-C/-i/-o/-v, less/more.

mkdir -p foo && cd foo

pwd                          # confirm where you are (CWD)
touch lesson-1.txt
echo hi > lesson-1.txt
cat lesson-1.txt
cp lesson-1.txt lesson-2.txt
rm lesson-1.txt lesson-2.txt

echo
echo "Hidden files start with a dot and are skipped by plain ls:"
touch .lesson.txt
ls                           # .lesson.txt is not listed
ls -a                        # now it shows up
ls -l .lesson.txt
rm .lesson.txt

echo
echo "grep context flags — build a small file to search:"
printf 'a\nb\nc\n' > file.txt
grep -A1 b file.txt          # 1 line of context After the match
grep -i B file.txt           # case-Insensitive
grep -o b file.txt           # Only print the matched text
grep -v b file.txt           # Invert — show non-matching lines
rm file.txt

echo
echo "man rm shows -i (confirm before delete) — worth aliasing:"
echo "  alias rm='rm -i'"

cd .. && rm -rf foo
```

02 Man Pages, Variables & Permissions

Learn to read a command's manual instead of guessing, tell the difference between programs and shell builtins, assign your first variables, and understand the read/write/execute permission bits that gate what a file can do.

▼ View script

○○○ bash-scripting/scripts/02-man-pages-variables-permissions.sh

```
#!/usr/bin/env bash
#
# Chapter 02 — Man Pages, Variables & Permissions
# Real commands: man, help, compgen -b, which, type, file, $PATH,
# built-in variables, chmod, stat.

echo "man covers real programs on disk; bash builtins have their own help:"
echo "  man cp        # a real program"
echo "  help history  # history is a shell builtin, not a program"
echo "  compgen -b    # lists every builtin"

echo
echo "which finds programs on \$PATH; it fails on builtins — type works on both:"
which ls
type history

echo
echo "file tells you what kind of file something is:"
file /usr/bin/ls 2>/dev/null || file /bin/ls

echo
echo "\$PATH is a colon-separated list — split it onto separate lines:"
echo "$PATH" | tr ':' '\n' | head -5

echo
echo "A few variables bash sets for you:"
echo "PWD=$PWD"
echo "USER=$USER"
echo "SHELL=$SHELL"

echo
echo "Capture a command's output into a variable:"
now=$(date)
echo "now=$now"

echo
echo "Permissions — chmod +x is what makes a script directly runnable:"
touch demo-script.sh
stat demo-script.sh | head -4
chmod +x demo-script.sh
ls -lha demo-script.sh
rm demo-script.sh
```

03 Finally Scripting

Your first real scripts: reading user input, packaging logic into functions, branching with if/else, repeating work with for loops, and redirecting input and output between commands and files.

▼ View script

○○○ bash-scripting/scripts/03-finally-scripting.sh

```
#!/usr/bin/env bash
#
# Chapter 03 — Finally Scripting
# Real commands: read -p, positional args ($1, $@), functions with
# local, if/[[ ]], for loops (list, range, C-style), I/O redirection.

echo "Take input from the user, or fall back to it if no argument was given:"
if [[ -n $1 ]]; then
    name=$1
else
    name='course-student'   # in place of an interactive read -p prompt
fi
echo "hello $name!"

echo
echo "A function using 'local' so it doesn't leak into the caller's scope:"
my-func() {
    local arg1=$1
    echo "hello from my function, arg1 is $arg1"
}
my-func "$@"

echo
echo "Return codes are separate from printed output:"
check-even() {
    (( $1 % 2 == 0 ))
}
check-even 4 && echo "4 is even (exit code: $?)"

echo
echo "The three for-loop shapes covered in this lesson:"
for name in foo bar baz; do echo "name is $name"; done
for c in {a..c}; do echo "c is $c"; done
for ((i = 0; i < 3; i++)); do echo "i is $i"; done

echo
echo "Reading lines piped in from another command:"
printf 'one\ntwo\n' | while read -r line; do
    echo "got line: $line"
done
```

04 Case Statements, Arrays & Substitution

Case statements offer a cleaner alternative to long if/elif chains. This chapter also introduces indexed and associative arrays, the IFS variable that controls word-splitting, command substitution, arithmetic expressions, and process substitution.

▼ View script

○○○ bash-scripting/scripts/04-case-arrays-substitution.sh

```
#!/usr/bin/env bash
#
# Chapter 04 — Case Statements, Arrays & Substitution
# Real commands: case/esac, indexed & associative arrays, IFS,
# $(...) command substitution, $((...)) arithmetic, <(...) process sub.

for name in dave bill anyone; do
    case "$name" in
        d*)  echo "matched d*: $name" ;;
        b*)  echo "matched b*: $name" ;;
        *)   echo "matched *: $name"  ;;
    esac
done

echo
echo "Indexed array — note *the difference* between [*] and [@]:"
array=(foo bar baz)
echo "element 0: ${array[0]}"
echo "last element: ${array[-1]}"
echo "all (as one word): ${array[*]}"
echo "all (as separate words): ${array[@]}"

echo
echo "Associative array — string keys instead of positions:"
declare -A capitals=([France]=Paris [Japan]=Tokyo)
for key in "${!capitals[@]}"; do
    echo "$key is ${capitals[$key]}"
done

echo
echo "Command substitution — prefer \$(...) over legacy backticks:"
thing=$(whoami)
echo "thing is $thing"

echo
echo "Arithmetic expression:"
i=0
((i++))
echo "i is $i"
echo "2 + 2 = $((2 + 2))"

echo
echo "Process substitution treats a command's output like a file:"
diff <(printf 'a\nb\n') <(printf 'a\nc\n') || true
```

05 Text Processing

The classic Unix text tools: cut for pulling out fields, tr for character translation, sed for stream editing, awk for column-based processing, grep for filtering by pattern, and find for locating files across a directory tree.

▼ View script

○○○ bash-scripting/scripts/05-text-processing.sh

```
#!/usr/bin/env bash
#
# Chapter 05 — Text Processing
# Real commands: cut, tr, sed, awk -F, grep -c, find -type/-name/-exec.

echo "name,age,city" > people.csv
echo "Alice,30,Boston" >> people.csv
echo "Bob,25,Denver" >> people.csv

echo "cut -d',' -f2 — pull just the age column:"
cut -d',' -f2 people.csv

echo
echo "sed — swap Boston for a placeholder:"
sed 's/Boston/[city]/' people.csv

echo
echo "awk -F',' — print field 1 and field 3, skipping the header:"
awk -F',' 'NR > 1 { print $1, $3 }' people.csv

echo
echo "grep -c — count matching lines instead of printing them:"
grep -c 'Alice' people.csv

rm people.csv

echo
echo "find — locate files by type and name, then act on each match:"
find . -maxdepth 1 -type f -name "*.sh"
find . -maxdepth 1 -type f -name "*.sh" -exec echo "found: {}" \;
```

06 Bash Arguments, Pipe Status & Timing

Read arguments passed into your script, inspect the exit status of every stage in a pipeline with PIPESTATUS (not just the last one), and measure how long a command takes to run.

▼ View script

○○○ bash-scripting/scripts/06-arguments-pipes-timing.sh

```
#!/usr/bin/env bash
#
# Chapter 06 — Bash Arguments, Pipe Status & Timing
# Real commands: bash -x (trace), bash -n (syntax check), bash -u
# (error on unset vars), PIPESTATUS, time.

cat > sample.sh << 'INNER_EOF'
#!/usr/bin/env bash
first_name='john'
last_name='eddy'
echo "Hello $first_name $last_name!"
INNER_EOF

echo "bash -n only checks syntax, it does not run the script:"
bash -n sample.sh && echo "syntax OK"

echo
echo "bash -x traces every command as it executes:"
bash -x sample.sh

rm sample.sh

echo
echo "PIPESTATUS holds the exit code of every stage in a pipeline, not"
echo "just the last one that \$? would give you:"
true | false | true
echo "PIPESTATUS: ${PIPESTATUS[@]}"

echo
echo "time reports real/user/sys time for a command:"
time sleep 0.2
```

07 Sourcing Code, Subshells & Return vs. Output

Sourcing loads another file's code into your current shell, while curly braces { } and parentheses ( ) control whether commands run in your current shell or an isolated subshell. Also covers the difference between a function's numeric return status and its printed output.

▼ View script

○○○ bash-scripting/scripts/07-sourcing-and-subshells.sh

```
#!/usr/bin/env bash
#
# Chapter 07 — Sourcing Code, Subshells & Return vs. Output
# Real commands: source, { } vs ( ), capturing stdout vs stderr
# separately when calling a function.

cat > lib.sh << 'INNER_EOF'
greet() {
    local name=$1
    echo "hello $name!"
}
INNER_EOF

source lib.sh      # loads greet() into THIS shell
greet dave
rm lib.sh

echo
echo "{ } runs in the current shell; ( ) runs in an isolated subshell:"
{ set_in_brace="yes"; }
echo "set_in_brace=$set_in_brace"
( set_in_paren="yes" )
echo "set_in_paren='${set_in_paren:-unset}'"

echo
echo "A function's stdout and stderr can be redirected independently:"
my-func() {
    echo "this goes to stdout" >&1
    echo "this goes to stderr" >&2
}
output=$(my-func 2>/dev/null)   # keep stdout, throw away stderr
echo "captured: $output"
```

08 Parameter & Array Expansion

Bash's parameter expansion syntax packs a lot of power into a small space: default values for unset variables, trimming prefixes and suffixes off strings, search-and-replace, and slicing arrays by index.

▼ View script

○○○ bash-scripting/scripts/08-parameter-and-array-expansion.sh

```
#!/usr/bin/env bash
#
# Chapter 08 — Parameter & Array Expansion
# Real commands: case conversion (^ , ,), default/required values
# (:-  ?), replacements (# ## % // ), substrings, array slicing.

s='dave eddy'
echo "uppercase first letter: ${s^}"
echo "uppercase everything:   ${s^^}"

echo
name=${1:-dave}
echo "default if unset: $name"

path=/home/dave/data.txt
echo "strip shortest prefix:  ${path#*/}"
echo "strip longest prefix:   ${path##*/}"
echo "strip shortest suffix:  ${path%/*}"
echo "replace first match:    ${path/a/o}"
echo "replace all matches:    ${path//a/o}"

echo
echo "Substrings by offset and length:"
echo "${s:0:4}"
echo "${s: -4}"

echo
echo "Loop a string one character at a time using substring expansion:"
len=${#s}
for ((i = 0; i < len; i++)); do
    printf '%s' "${s:i:1}"
done
echo

echo
echo "Array expansion works the same way, applied to every element:"
arr=(foo bar baz)
printf '<%s>\n' "${arr[@]/a/1}"
```

09 Globbing

Wildcards like * and ? let you match filenames without listing them one by one. This chapter adds extended globbing for more expressive patterns and shell options like nullglob that change how a non-matching glob behaves.

▼ View script

○○○ bash-scripting/scripts/09-globbing.sh

```
#!/usr/bin/env bash
#
# Chapter 09 — Globbing
# Real commands: *, ?, [..] character classes, extglob's !( ) +( ) @( ),
# and the nullglob/dotglob/globstar shell options.

mkdir -p files
touch files/foo.txt files/foo.jpg files/bar.txt files/baz.jpg

echo "Basic wildcard — every .txt file:"
printf '%s\n' files/*.txt

echo
echo "? matches exactly one character:"
printf '%s\n' files/ba?.txt

echo
echo "[..] matches one character from a set:"
printf '%s\n' files/ba[rz].txt

echo
echo "Extended globbing (needs extglob enabled) — everything except .txt:"
shopt -s extglob
printf '%s\n' files/!(*.txt)
shopt -u extglob

echo
echo "nullglob — a non-matching glob expands to nothing instead of the"
echo "literal pattern string:"
shopt -s nullglob
printf '<%s>\n' files/*.md
shopt -u nullglob

rm -rf files
```

10 Brace Expansion

Brace expansion generates text from comma-separated lists or numeric ranges before the shell even looks at the filesystem — a fast way to produce sequences of filenames or values.

▼ View script

○○○ bash-scripting/scripts/10-brace-expansion.sh

```
#!/usr/bin/env bash
#
# Chapter 10 — Brace Expansion
# Real commands: {a,b,c} comma lists, {1..N} numeric ranges, and
# combining braces with a shared filename prefix.

echo "Comma list:"
echo {hello,world}

echo
filename='my-file'
echo "Prefix + comma list, useful for generating related filenames:"
echo "$filename."{txt,jpg,mov}

echo
echo "Numeric range with a step:"
echo {1..20..5}

echo
echo "Combined with globbing — braces don't require the files to exist,"
echo "unlike a glob pattern:"
mkdir -p files
touch files/{foo,bar}.txt
printf '<%s>\n' files/{foo,bar}.txt
rm -rf files
```

11 printf, Date Formatting & Regular Expressions

printf gives you precise control over formatted output, the date command formats timestamps however you need, [[=~]] matches strings against regular expressions, and mapfile reads input straight into an array.

▼ View script

○○○ bash-scripting/scripts/11-printf-dates-regex.sh

```
#!/usr/bin/env bash
#
# Chapter 11 — printf, Date Formatting & Regular Expressions
# Real commands: printf format specifiers (%s %d %5s %-5s %05d),
# date +FORMAT, printf '%(FORMAT)T', regex with [[ =~ ]], mapfile.

echo "printf needs a format string — passing data directly is a bug:"
s='hello world'
printf '%s\n' "$s"

echo
echo "Width and padding:"
printf '%5s|%-5s|\n' hi hi
printf '%05d\n' 55

echo
echo "Date formatting, two ways:"
datefmt='%Y-%m-%d %H:%M:%S'
date "+$datefmt"
printf "%($datefmt)T\n" -1

echo
echo "Matching a pattern with a regex:"
email='user@example.com'
if [[ "$email" =~ ^[^@]+@[^@]+\.[a-z]+$ ]]; then
    echo "'$email' looks like an email"
fi

echo
echo "mapfile reads lines straight into an array:"
printf 'one\ntwo\nthree\n' > lines.txt
mapfile -t lines < lines.txt
echo "line 2: ${lines[1]}"
rm lines.txt
```

12 Test Operators

The classic [ ] test command, the bash-only [[ ]] with safer parsing, and the test command itself — plus the string and file-check flags (-z, -f, -d, and friends) you'll reach for constantly in conditionals.

▼ View script

○○○ bash-scripting/scripts/12-test-operators.sh

```
#!/usr/bin/env bash
#
# Chapter 12 — Test Operators
# Real commands: help '[[', help '[', help test, and quoted
# ANSI-C ($'...') strings.

touch demo.txt

[ -f demo.txt ]; echo "[ -f ] exit code: $?"
[[ -f demo.txt && -r demo.txt ]] && echo "[[ ]]: demo.txt exists and is readable"
test -f demo.txt && echo "test: demo.txt exists"

rm demo.txt

echo
echo "Plain echo doesn't interpret backslash escapes by default:"
echo 'line1\nline2'

echo
echo "\$'...' (ANSI-C quoting) does interpret them:"
echo $'line1\nline2'
```

13 Trap Signals & Named Pipes

trap intercepts signals like Ctrl+C so your script can clean up before exiting, and named pipes (FIFOs) let two separate processes pass data through a file-like path on disk.

▼ View script

bash-scripting/scripts/13-traps-and-named-pipes.sh

```
#!/usr/bin/env bash
#
# Chapter 13 — Trap Signals & Named Pipes
# Real commands: trap ... exit, mkfifo, reading/writing a FIFO path.

cleanup() {
    echo "cleanup function running"
}
trap cleanup exit

echo "script starting"
echo "..."
echo "script done"
# 'cleanup' fires here automatically because of the trap on exit

echo
echo "Named pipes let two processes talk through a path on disk:"
pipe="/tmp/course-pipe-$$"
mkfifo "$pipe"

echo "[client] hello" > "$pipe" &
read -r line < "$pipe"
echo "[reader] got: $line"

rm -f "$pipe"
```

14 Terminal Output & Color

ANSI escape codes add color and move the cursor around for progress indicators, and checking whether stdout is a real TTY keeps your script from dumping raw escape codes into a log file or pipe.

▼ View script

○○○ bash-scripting/scripts/14-terminal-output-and-color.sh

```
#!/usr/bin/env bash
#
# Chapter 14 — Terminal Output & Color
# Real commands: \e[...m ANSI escapes, tput setaf/sgr0, cursor
# save/restore with \e7 and \e8, and the [ -t 1 ] TTY check.

echo -e '\e[1mbold text\e[0m'
echo $'\e[31mred text\e[0m'

echo
echo "tput is a more portable way to reach the same codes:"
tput setaf 2; echo "green via tput"; tput sgr0

echo
echo "Save cursor position, print elsewhere, then restore it:"
printf '\e7'
printf '\e[5;10Hoverwritten text'
printf '\e8'
echo

echo
echo "Only use color if stdout is an actual terminal:"
if [[ -t 1 ]]; then
    echo "stdout is a TTY — color is safe here"
else
    echo "stdout is redirected — skip the escape codes"
fi
```

15 PS1 & Customizing Bash

Your shell prompt is just a variable (PS1) you can customize to show your user, host, and directory, and readline shortcuts like Ctrl+R for history search make everyday terminal use faster.

▼ View script

○○○ bash-scripting/scripts/15-prompt-and-customization.sh

```
#!/usr/bin/env bash
#
# Chapter 15 — PS1 & Customizing Bash
# Real commands: PS1 escape sequences, shopt -s cdspell/autocd,
# conditional aliases for ~/.bashrc.

echo "Current PS1:"
echo "$PS1"

echo
echo "A custom prompt using \u \h \w — user, host, working directory:"
echo '  PS1="\u@\h:\w\$ "'

echo
echo "Two interactive-only shell options worth turning on in .bashrc:"
echo '  shopt -s cdspell   # forgives small typos in cd targets'
echo '  shopt -s autocd    # typing a directory name alone cd'"'"'s into it'

echo
echo "A conditional alias — only defined if the feature is supported:"
echo '  ls --color=auto &>/dev/null && alias ls="ls --color=auto"'

echo
echo "Readline shortcuts worth practicing at the prompt (not scriptable):"
echo "  Ctrl+W   delete one word back"
echo "  Ctrl+U   delete to start of line"
echo "  Ctrl+R   reverse search through history"
```

16 Common Pitfalls

Field notes from real bugs: why parsing 'ls' output breaks on odd filenames, why aliases can't take arguments the way functions can, and why string length can surprise you with multibyte characters.

▼ View script

○○○ bash-scripting/scripts/16-common-pitfalls.sh

```
#!/usr/bin/env bash
#
# Chapter 16 — Common Pitfalls
# Real commands: the classic `for f in $(ls)` bug, alias argument
# limitations, and ANSI codes inflating string length.

mkdir -p files
touch files/a files/b "files/d      e"

echo "The fragile way — breaks on filenames containing spaces:"
for f in $(ls files); do
    echo "  bad: $f"
done

echo
echo "The safe way — glob directly, no ls, no word-splitting:"
for f in files/*; do
    echo "  good: $f"
done

rm -rf files

echo
echo "Aliases can't take positional arguments the way functions can:"
shopt -s expand_aliases
alias greet-alias='echo hello \$1'   # \$1 never gets replaced
greet-alias world

greet-func() { echo "hello $1"; }
greet-func world

echo
echo "ANSI color codes count toward string length even though they're"
echo "invisible on screen:"
colored=$'\e[33mHello\e[0m'
echo "visible text is 5 characters, but \${#colored} reports: ${#colored}"
```

17 Forkbombs & Wrap-up

A look at the infamous forkbomb one-liner as a cautionary piece of shell trivia — explained, never executed — followed by a recap of everything the course covered.

▼ View script

○○○ bash-scripting/scripts/17-forkbombs-and-wrapup.sh

```
#!/usr/bin/env bash
#
# Chapter 17 — Forkbombs & Course Wrap-up
#
# This chapter covers the classic bash forkbomb: a function that
# recursively spawns two copies of itself in the background, forever,
# exhausting process slots or memory. It's a well-known piece of shell
# trivia (search "bash fork bomb" for the one-liner), included here
# for recognition only — this file deliberately does not include or
# execute one.
#
# If you ever want to see the effect safely, do it only in a disposable
# VM or container, and cap your process count first:
#   ulimit -u 100

echo "A forkbomb defines a function that calls itself twice, piped"
echo "together and backgrounded, doubling the process count on every"
echo "generation until the system runs out of room."

echo
echo "That wraps up Part 1 — Bash Scripting."
echo "Next up: Part 2 — R Programming."
```

02

R Programming

17 chapters, from installing R to a first tour of PCA, clustering, and regression. Course content adapted from an introductory R course by Dr. Lema Logamou Seknewna (AIMS / National Institute of Statistics of Rwanda).

00 Getting Started

Installing R and RStudio, setting up R Markdown so it can render to HTML, Word, and PDF, and getting the handful of extra tools (like tinytex) that later chapters depend on.

▼ View R Markdown

○○○ r-programming/chapters/00-getting-started.Rmd

````
---
title: "00 - Getting Started with R"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

## What is R?

R is a statistical programming language created in 1992 by Ross Ihaka and
Robert Gentleman at the University of Auckland, New Zealand, and first
released as version 1.0 in February 2000. Its name comes from the initials
of its creators' first names. R descends from the S language, which was
developed at Bell Laboratories in the mid-1970s. See the
[official R project page](https://www.r-project.org/about.html) for more
history.

## Install R

Download the build for your operating system from CRAN:

* [Windows](https://cloud.r-project.org/bin/windows/base/)
* [macOS](https://cloud.r-project.org/bin/macosx/)
* [Linux (Debian, Fedora/Red Hat, Ubuntu)](https://cloud.r-project.org/bin/linux/)

## Install RStudio

RStudio (now Posit) is the IDE most R users work in day to day. Download it
from [posit.co/download/rstudio-desktop](https://posit.co/download/rstudio-desktop/),
or use [Posit Cloud](https://posit.cloud/) to run RStudio entirely in a
browser with no install at all.

## Setting up R Markdown

### Required packages

* **tinytex** — a lightweight LaTeX distribution needed to render R Markdown
  documents to PDF.

```{r eval=FALSE}
if (!"tinytex" %in% rownames(installed.packages())) {
  install.packages("tinytex")
}
library(tinytex)
install_tinytex() # installs LaTeX itself
```

* **Rtools** (Windows only) — needed to build packages from source. Download
  from [cran.r-project.org/bin/windows/Rtools](https://cran.r-project.org/bin/windows/Rtools/).

Once these are installed, R Markdown (`.Rmd`) files like the ones in this
course can render to HTML, Word, or PDF directly from RStudio's Knit button.
````

01 Operators

Arithmetic operators, logical operators, and value-matching with match() and %in% — the small building blocks that every later script leans on.

▼ View R Markdown

○○○ r-programming/chapters/01-operators.Rmd

````
---
title: "01 - Operators"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

R can act as a calculator directly from the console — every operator below
can be typed and evaluated on the spot.

# Arithmetic Operators

### Addition: `+`
```{r}
1 + 1
```

### Subtraction: `-`
```{r}
1 - 1
```

### Multiplication: `*`
```{r}
1 * 1
```

### Division: `/`
```{r}
1 / 1
```

### Modulus (remainder of a division): `%%`
```{r}
1 %% 2
```

### Exponent: `^` or `**`
```{r}
2 ^ 10 # or 2 ** 10
```

### Integer division: `%/%`
```{r}
````
````

```
1035 %/% 3
```

Logical Operators

Less than: `<`
```{r}
1 < 1
```

Less than or equal to: `<=`
```{r}
1 <= 1
```

Greater than: `>`
```{r}
1 > 1
```

Greater than or equal to: `>=`
```{r}
1 >= 1
```

Exactly equal to: `==`
```{r}
"R" == "r"
```

R is case-sensitive — `"R"` and `"r"` are different values.

The equality operator can also compare one element against several at once:
```{r}
"Species" == c("Sepal.Length", "Sepal.Width", "Petal.Length", "Petal.Width", "Species")
```

Not equal to: `!=`
```{r}
1 != 1
```

Negation / NOT: `!`
Flips `TRUE` to `FALSE` and vice versa.
```{r}
!TRUE  # or !T
!FALSE # or !F

!(TRUE & FALSE) # this is TRUE
!(FALSE | TRUE) # this is FALSE
```

AND: `&`
```{r}
TRUE & TRUE
TRUE & FALSE
FALSE & FALSE
```

OR: `|`
```{r}
TRUE | TRUE
TRUE | FALSE
FALSE | FALSE
```

```
```

Value Matching

`match()` returns the position of the first match of its first argument in
its second argument. Check `?match` for the full documentation.
```{r}
match("Species", c("Sepal.Length", "Sepal.Width", "Petal.Length",
                   "Petal.Width", "Species"))
```

`%in%` checks whether a value exists anywhere in a vector, returning
`TRUE`/`FALSE` instead of a position:
```{r}
"Species" %in% c("Sepal.Length", "Sepal.Width", "Petal.Length",
                 "Petal.Width", "Species")
```
```

---

## 02 R Objects, Assignment & Data Types

Assigning values with <-, =, and ->, the rules for valid variable names, R's core data types (numeric, integer, complex, character, logical, raw, factor), and how to convert cleanly between them.

**▼ View R Markdown**

○○○ r-programming/chapters/02-objects-assignment-data-types.Rmd

```
---
title: "02 - R Objects, Assignment & Data Types"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

Assignment

R accepts three assignment operators: `<-`, `=`, and `->`.

A variable name:

* can begin with a letter or a dot. E.g. `a <- 1`, `0 -> .a`
* cannot contain spaces — use `_` or `.` instead
* can contain numbers, just not as the first character. E.g. `a1 <- 1`

```{r error=TRUE}
v rsion <- 4.3.2 # invalid: a space inside the name breaks the parser
```

```{r}
a <- 1
```

```r
b <- 2
0 -> .a
a1 <- .a
```

Data Types

R has these core data types: numeric, integer, complex, character, logical,
raw, and factor.

Numeric / double
Examples: `10.5`, `55`, `787`, `pi`.
```{r}
PI <- pi; class(PI); typeof(PI)
n <- 55; class(n); typeof(n)
```

Integer
Append `L` to a literal to declare it as an integer (`1L`, `55L`, `100L`).
```{r}
n <- 55L
class(n)
```

Complex
An example of a complex number is $9 + 3i$, where i is the imaginary part.
Multiplying a real number by `1i` converts it to complex.
```{r}
z <- 9 + 3i
class(z)
typeof(z)

z1 <- a + 1i * b
print(z1)
class(z1)
```

Character / string
```{r}
string <- "I am Learning R"
class(string)
```
Remember: `"LeaRning"` is different from `"Learning"` — R is case-sensitive.

Logical / Boolean
```{r}
TRUE  # or T
FALSE # or F
```
A logical value is often the *result* of a test:
```{r}
"LeaRning" == "Learning"
```

Raw
```{r}
text <- "I am learning R."
(raw_text <- charToRaw(text))
class(raw_text)
```
Converting raw back to text:
```{r}
rawToChar(raw_text)
```

Factors
Factors represent qualitative/categorical data — colors, pass/fail, movie ratings — and are the backbone of statistical modeling in R.
```{r}
Gender <- factor(c("Female", "Male"))
print(Gender)
class(Gender)
```

Logical, revisited
```{r}
v <- TRUE
w <- FALSE

class(v); typeof(v)
!v

isTRUE(w)
```

Creating objects directly

Numeric:
```{r}
n <- 10
x <- numeric(n) # a numeric vector of length n, filled with zeros
print(x)

x[1] <- 2.5
print(x)
```

Integer — note that assigning a decimal silently upgrades it to numeric:
```{r}
n <- 10
x <- integer(n)
print(x)
class(x)

x[1] <- 2.5 # R converts the whole vector to numeric
class(x)
print(x)
```

Conversions

To convert between data types, use one of the `as.*()` functions from base R. The naming convention is `as.` followed by the target type.

* Numeric to character
```{r}
n <- 55
n_char <- as.character(n)
print(n_char); class(n_char)
```

* Character to numeric
```{r}
s <- "123"
s_num <- as.numeric(s)
print(s_num); class(s_num)
```
A string that isn't a valid number becomes `NA` (with a warning) rather

````
than causing an error:
```{r}
suppressWarnings(as.numeric("abc"))
```

* Factor to character
```{r}
f <- factor(c("low", "high", "medium"))
f_char <- as.character(f)
print(f_char); class(f_char)
```

* Character to factor
```{r}
level <- "medium"
level_fct <- as.factor(level)
print(level_fct); class(level_fct)
```
````

---

03 Data Structures I: Vectors & Matrices

Building vectors with seq(), rep(), and sample(), then moving into matrix algebra — transpose, inverse, solving linear systems, and eigenvalues — capped off with a hand-rolled PCA on the iris dataset.

▼ View R Markdown

○○○ r-programming/chapters/03-vectors-and-matrices.Rmd

````
---
title: "03 - Data Structures I: Vectors & Matrices"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

R's most-used data structures are vectors, matrices, arrays, lists, factors,
and data frames. This chapter covers the first two.

# Scalars and Vectors (1D)

A scalar is any single number; a vector is a collection of values of the
same type (and can represent a sequence).

Mixing types inside `c()` forces R to coerce everything to one common type:
```{r}
v <- c(1, "R", TRUE, FALSE, NA)
print(v)
class(v) # everything becomes character (NA is compatible with any type)
```
````

```{r}
v2 <- c(1, 4, 8, FALSE, TRUE, FALSE, FALSE, TRUE, "R" == "r")
print(v2)
class(v2) # everything becomes numeric: FALSE -> 0, TRUE -> 1
```

Sequences with `seq()`
```{r}
x <- seq(0, 2 * pi, length.out = 90)
length(x)
unique(round(diff(x), 5)) # the common step size

head(x, 5); tail(x)
range(x)
```

```{r}
diff(c(4, 6))
diff(range(x)) # same as max(x) - min(x)
```

Indexing with a non-integer floors to the nearest integer:
```{r}
x[11.8] == x[11]
as.integer(9.123486785) # what R does internally with a fractional index
```

The length of a vector:
```{r}
length(x)
a <- 9
length(a) # a scalar is just a vector of length 1
```

Repetition with `rep()`
```{r}
rep("I learn R", 3)
rep(c(0, 1), 5)
rep(c(0:1), c(3, 3))
rep(c(0:1), each = 3)
```

Random sampling
```{r}
set.seed(123) # fixes the randomness for reproducibility
y <- sample(0:1, size = 200, replace = TRUE, prob = c(1/3, 1 - 1/3))
sum(y == 0)   # count of zeros
sum(y == 1)   # count of ones
```

`table()` counts occurrences for you:
```{r}
table(y)
```

`prop.table()` converts counts to proportions:
```{r}
prop.table(table(y))
mean(y == 0) # a shortcut: mean() of a logical vector is a proportion
```

A small survey example

```{r}
```

```
Gender <- c("Female", "Male", "Male", "Male", "Female", "Female", "Male",
"Male", "Female", "Female", "Male", "Male", "Male", "Male", "Female",
"Female", "Male", "Female", "Female", "Male", "Female", "Male",
"Male", "Male", "Male", "Female", "Femaale", "Male", "Female",
"Male", "Male", "Female", "Male", "Female", "Malle", "Male", "Female",
"Female", "Femmale", "Female", "Female", "Female", "Female", "Male",
"Male", "Male", "Male")

sum(Gender == "Female")
mean(Gender == "Female")
table(Gender) # note the 3 misspelled entries
```

Cleaning up the typos:
```{r}
Gender[Gender == "Femaale"] <- "Female"
Gender[Gender == "Femmale"] <- "Female"
Gender[Gender == "Malle"]   <- "Male"
table(Gender)
```

Sub-setting with `sample()`:
```{r}
set.seed(76)
table(sample(Gender, size = 20))
```

Vector arithmetic
```{r}
k <- 2
v <- c(1, 0, 3)

k + v; v + k
k - v; v - k
k * v; v * k
v / k
k / v
```

```{r}
k <- 1/2
v <- c(pi, 0, 1, 4)
w <- c(0, pi, pi, 0, 0)

k * v; v * k
v ^ k
v + w
```

Matrices (2D)

```{r}
(A <- matrix(1:25, ncol = 5, nrow = 5)) # column-major fill by default
(B <- matrix(1:25, nrow = 5, ncol = 5, byrow = TRUE))
```

Building a matrix by hand
```{r}
v <- c(1, rep(0, 2), 1, 0, rep(1, 3), rep(0, 2), rep(1, 2))
A <- matrix(v, 3, byrow = TRUE)
print(A)
dim(A)
```

The same result using `dim<-`:
```{r}
v <- c(1, rep(0, 2), 1, 0, rep(1, 3), rep(0, 2), rep(1, 2))
dim(v) <- c(4, 3)
v <- t(v)
v
```

A zero matrix:
```{r}
n <- 6; p <- 8
matrix(0, nrow = n, ncol = p)
```

```{r}
(A <- matrix(c(1, 0, 2, 5, 2, 1, 4, 2, 0), nrow = 3))
(B <- matrix(c(2, 5, 2, 3, 1, 1, 0, 1, 1), nrow = 3))
```

Matrices from vectors: `cbind` / `rbind`
```{r}
v1 <- c(1, 0, 2); v2 <- c(5, 2, 1); v3 <- c(4, 2, 0)
(M1 <- cbind(v1, v2, v3))
(M2 <- rbind(v1, v2, v3))
```

Matrix operations

Transpose:
```{r}
(A_T <- t(A))
```

Addition / subtraction:
```{r}
A + B
A - B
```

Multiplication (`%*%`, not `*`):
```{r}
ncol(A) == nrow(B)
A %*% B
```

Inverse:
```{r}
(A_inv <- solve(A))
round(A %*% A_inv) # sanity check: should be the identity matrix
```

Solving a system of equations

$$
\begin{cases}
2x + 2y &= 0\\
x + 3y &= 2
\end{cases}
$$

```{r}
A1 <- matrix(c(2, 2, 1, 3), nrow = 2, byrow = TRUE)
b <- c(0, 2)
solve(A1, b)
```

```

## Eigenvalues / eigenvectors

Eigenvalues $\lambda$ of a square matrix $A$ solve the characteristic
equation $\det(A - \lambda I_n) = 0$. They're the mathematical basis of
[Principal Component Analysis](https://www.datacamp.com/tutorial/pca-analysis-r),
covered in Chapter 15.

```{r}
ev <- eigen(A)
print(ev$values)

P <- ev$vectors
round(solve(P) %*% A %*% P, 4) # recovers a diagonal matrix of eigenvalues
```

Worked exercise: manual PCA on `iris`

1. Remove the last column (`Species`) from `iris`:
```{r}
data(iris)
dta <- iris[, -5]
```

2. Compute the average of each remaining column:
```{r}
avg <- colMeans(dta)
print(avg)
```

3. Center the data by subtracting each column's average:
```{r}
scaled_dta <- scale(dta, center = TRUE, scale = FALSE)
head(scaled_dta, 3)
```

4. Eigen-decompose `t(scaled_dta) %*% scaled_dta`:
```{r}
cov_mat <- t(scaled_dta) %*% scaled_dta
ev <- eigen(cov_mat)
ev$values
```

5. Project onto the first two eigenvectors:
```{r}
P <- ev$vectors[, 1:2]
projection <- scaled_dta %*% P
head(projection)
```

This is exactly what `prcomp()` does under the hood:
```{r}
pca <- prcomp(dta, center = TRUE, scale. = FALSE)
head(pca$x[, 1:2])
```

---

04 Data Structures II: Arrays, Lists & Data Frames

Multi-dimensional arrays, lists that hold mixed types together, and data frames — including
```

turning a continuous age variable into readable age-group categories.

▼ View R Markdown

○○○ r-programming/chapters/04-arrays-lists-dataframes.Rmd

````
---
title: "04 - Data Structures II: Arrays, Lists & Data Frames"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

# Arrays

Arrays generalize matrices to more than two dimensions.
```{r}
(aRray <- array(1:24, dim = c(3, 4, 2)))
class(aRray)
```

A classic use case is gridded (NetCDF-style) data: longitude as columns,
latitude as rows, and time as the third dimension — each time slice being
its own row x column matrix.
```{r}
dim(aRray)
aRray[1, 1, 2] # row 1, column 1, of the 2nd matrix "slice"
```

# Lists

A list holds objects of different types and sizes together.
```{r}
L <- list()
A <- matrix(0, 5, 6)

L$A <- A
L$my_seq <- sample(10, 5)
L$Bool <- TRUE
L[["iris_head"]] <- head(iris, 3)
length(L)

L[[5]] <- "I am learning R"
names(L)
names(L)[5] <- "String"
names(L)
```

```{r}
mylist <- list("matrix" = A,
               "sequence" = 1:5,
               "Bool" = TRUE)
mylist$matrix
class(mylist[[1]])
````
````

```
mylist[c("matrix", "Bool")]
```

Data Frames

A data frame is a table of n rows (observations) by p columns
(variables), where each column can hold a different data type.

Converting a continuous variable into categories
```{r}
set.seed(123)
n <- 120
df <- data.frame(
  "ID"  = paste0("Particip_", 1:n),
  "age" = sample(0:120, size = n)
)
head(df, 5)
```

```{r}
brks <- seq(0, 120, by = 10)
df$age_groups <- cut(df$age, breaks = brks, include.lowest = TRUE)
head(df, 5)
```

Exercise: summarizing the age groups

1. Counts by age group:
```{r}
(counts <- table(df$age_groups))
```

2. A barplot of the age groups:
```{r fig.height=4, fig.width=7}
x <- barplot(counts, col = "purple", main = "Distribution of ages", xaxt = "n")
text(x = x, y = -1.5, cex = 0.8, xpd = TRUE, srt = 45, labels = names(counts))
```

3. A histogram of raw ages:
```{r fig.height=4, fig.width=7}
hist(df$age, main = "Histogram of ages", xlab = "age", col = "pink", breaks = 12)
```

4. IDs of participants in the (70, 80] age group:
```{r}
df$ID[df$age_groups == "(70,80]"]
```

```{r}
l <- split(df, df$age_groups)
names(l)
l$`(70,80]`$ID
```

Creating a data frame directly
```{r}
df2 <- data.frame(x = rnorm(10), y = rpois(10, 2))
head(df2, 3)
```

---

05 **Data Cleaning: Missing Values & Outliers**

Finding and filling NAs, recovering numbers trapped in messy strings, and flagging outliers with the IQR method before imputing them away.

▼ View R Markdown

○○○ r-programming/chapters/05-data-cleaning.Rmd

````
---
title: "05 - Data Cleaning: Missing Values & Outliers"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

# Missing Values (`NA`)

```{r}
x <- c(NA, 1, 2, NA, 3, NA, 3.55)
which(is.na(x))  # which positions are missing
which(x >= 2)    # which positions are >= 2 (NAs drop out silently)
which(!is.na(x)) # which positions are NOT missing
```

Replacing missing values with the mean of the non-missing ones:
```{r}
x[is.na(x)] <- mean(x, na.rm = TRUE)
print(x)
```

## NAs introduced by coercion

Converting a messy character vector straight to numeric turns anything
non-numeric into `NA`:
```{r}
x <- c("2", "1", "2", "7", "$8", "3", "2.5", "9", "2,7")
y <- suppressWarnings(as.numeric(x))
print(y)

id <- which(is.na(y))
id
```

Cleaning the offending values with `gsub()` before converting:
```{r}
x <- c("2", "1", "2", ",7", "$8", "3", "&2.5", "9", "2,7")
x <- gsub("\\$|\\&", "", x)   # drop currency symbols
x <- gsub(",", ".", x)        # normalize decimal commas to dots
y <- as.numeric(x)
print(y)
```

# Outlier Detection

## The Inter-Quartile Range (IQR) method
````

A value outside the interval below is treated as an outlier:

$$[Q_1 - 1.5\times IQR,\ Q_3 + 1.5\times IQR]$$

where Q_1/Q_3 are the 25th/75th percentiles and $IQR = Q_3 - Q_1$.

```{r fig.height=4, fig.width=5}
x <- c(19.103, 19.632, 22.494, 20.113, 10.5, 22.744, 20.737, 17.976,
       18.901, 192.87, 21.959, 0.001, 20.641, 20.177, 19.111, 22.859,
       207.97, 16.853, 21.122, 19.244, 18.291, 19.651, 18.358, 18.834, 19)

x_copy <- x
boxplot(x_copy, main = "Before handling outliers")
```

```{r}
Q <- as.numeric(quantile(x, probs = c(0.25, 0.5, 0.75)))
d <- as.data.frame(t(Q))
names(d) <- paste0("Q", 1:length(Q))
d$IQR      <- d$Q3 - d$Q1
d$LowerLim <- d$Q1 - 1.5 * d$IQR
d$UpperLim <- d$Q3 + 1.5 * d$IQR
d
```

Flagging every value as an outlier or not:
```{r}
(d_out <- data.frame(x = x, outlier = x < d$LowerLim | x > d$UpperLim))
```

Two equivalent ways to pull out just the outliers:
```{r}
x[d_out$outlier]                                  # using our own table
(bxpt <- boxplot(x, range = 1.5, plot = FALSE))$out # using boxplot()'s own $out
```

Imputing the outliers

Treat outliers as missing, then fill them with the mean of everything else:
```{r}
x[d_out$outlier] <- NA
x[is.na(x)] <- mean(x, na.rm = TRUE)
```

```{r fig.height=4, fig.width=7}
par(mfrow = c(1, 2))
boxplot(x_copy, main = "Before\nhandling outliers", cex.main = 0.8)
boxplot(x, main = "After\nhandling outliers", col = "green", cex.main = 0.8)
```

No more outliers remain after imputation.

---

06 Charts & Distribution Simulations

Base R plotting — bar, pie, histogram, scatter — then simulating uniform, binomial, Gaussian, exponential, and Poisson distributions, plus fitting and interpreting a simple linear model.

▼ View R Markdown

○○○ r-programming/chapters/06-charts-and-distributions.Rmd

````
---
title: "06 - Charts & Distribution Simulations"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

# Base R Charts

## Bar chart
```{r fig.height=4, fig.width=7}
frequencies <- c(Female = 35, Children = 25, Male = 40)
barplot(frequencies, cex.names = 0.8,
        names.arg = names(frequencies),
        col = c("purple", "salmon", "pink"))
```
`cex.names` shrinks the axis labels; lower it further if labels get cut off.

## Pie chart
```{r fig.height=4, fig.width=8}
par(mfrow = c(1, 2))
pie(frequencies, cex = 0.8,
    labels = paste0(names(frequencies), " (", frequencies, ")"),
    col = c("purple", "salmon", "pink"))
pie(frequencies, cex = 0.8,
    labels = paste0(names(frequencies), " (", frequencies, ")"),
    col = rainbow(length(frequencies)))
```

## Histograms and density
```{r fig.height=4, fig.width=8}
set.seed(12092024)
x <- sample(1:120, size = 1000, replace = TRUE)
par(mfrow = c(1, 2))
hist(x, probability = TRUE) # densities instead of raw counts
plot(density(x), col = "salmon", main = "density of x")
```

```{r fig.height=4, fig.width=6}
with(iris, hist(Sepal.Length, probability = TRUE))
lines(density(iris$Sepal.Length), col = "red")
```

## Scatter plot
```{r fig.height=4, fig.width=6}
x <- seq(0, 2 * pi, length.out = 50)
y <- sin(x)
plot(x, y, pch = 16, col = "blue")
```

**Exercise:** scatter plot of `Petal.Length` vs `Petal.Width`, colored by
species, with a fitted regression line.
```{r fig.height=4, fig.width=6}
f <- Petal.Length ~ Petal.Width
````
````

```r
lr <- lm(f, data = iris)
plot(f, data = iris, col = iris$Species, pch = 16)
abline(lr, col = "blue", lwd = 2, lty = 3)
legend("topleft", legend = unique(iris$Species),
       col = unique(iris$Species), pch = 16, bty = "n")
```

Distribution Simulations

Uniform distribution

$$
f_X(x) = \begin{cases} \frac{1}{b-a} & \text{if } x \in[a, b] \\
0 & \text{else}\end{cases}
$$

```{r fig.height=4, fig.width=6}
set.seed(123)
x <- runif(1000)
hist(x, probability = TRUE, xlab = NULL,
     main = "Histogram of a uniform distribution", col = "turquoise")
lines(density(x), col = "red", lwd = 2, lty = 3)
abline(v = mean(x), col = "blue", lty = 2, lwd = 2)
legend("topleft", lty = c(2, 2), col = c("red", "blue"),
       legend = c("Density", "Average"))
```

Testing whether a sample looks uniform vs. normal:
```{r fig.height=4, fig.width=5}
ks.test(x, "punif") # p > 0.05 supports "looks uniform"
qqnorm(x)
qqline(x, col = "red")
```

Binomial distribution

$$
P(X = k) = {n \choose k}p^k(1-p)^{n-k}
$$

```{r fig.height=4, fig.width=8}
rbinom_dist <- rbinom(n = 10000, size = 1, prob = 0.3)
par(mfrow = c(1, 2))
pie(table(rbinom_dist))
barplot(table(rbinom_dist))
```
A histogram isn't well suited to discrete data like this – bar/pie charts
work better.

Gaussian (normal) distribution

$$
f_X(x) = \frac{1}{\sigma\sqrt{2\pi}}\exp\left\{-\frac{1}{2\sigma^2}(x-\mu)^2\right\}
$$

```{r fig.height=4, fig.width=6}
set.seed(13092024)
gauss_dist <- rnorm(1000, mean = 0, sd = 1)
hist(gauss_dist, probability = TRUE, breaks = 30, xlab = NULL,
     main = "Histogram of a standard normal distribution", col = "turquoise")
lines(density(gauss_dist), col = "red", lwd = 2, lty = 2)
abline(v = mean(gauss_dist), col = "blue", lty = 2, lwd = 2)
```

```{r}
mean(gauss_dist)
sd(gauss_dist)
```

Simulating a linear relationship

$$y_i = \alpha + \beta x_i + \varepsilon_i,\quad \varepsilon_i \sim \mathcal{N}(0,1)$$

```{r}
set.seed(123)
x <- rnorm(1000, mean = 5, sd = 1.5)
error_term <- rnorm(1000)
z <- 4 + 8 * x + error_term # true relationship, known only because we simulated it

df <- data.frame(x, z)
head(df, 5)
```

Fitting a simple linear model:
```{r fig.height=5, fig.width=7}
LR <- lm(z ~ x, data = df)
coefs <- coefficients(LR)
summary(LR)

par(mfrow = c(2, 2))
plot(LR) # standard diagnostic plots
```

Estimated model: $\hat{y} =$ `r round(coefs[[1]], 2)` + `r round(coefs[[2]], 2)`
x. Predicting for a new value, $x = 15$:
```{r}
new_data <- data.frame(x = 15)
(new_data$predicted <- predict(object = LR, newdata = new_data))
```

```{r fig.height=4, fig.width=6}
plot(predicted ~ x, data = new_data, col = "darkgreen", xlim = c(0, 15),
     ylim = c(0, new_data$predicted), pch = 16, ylab = "z")
points(z ~ x, data = df, col = "darkorange", pch = 20)
abline(LR, col = "blue")
legend("topleft", c("Actual", "Predicted"),
       col = c("darkorange", "darkgreen"), pch = c(20, 16), bty = "n")
```

Exponential distribution
```{r fig.height=4, fig.width=6}
set.seed(13092024)
exp_dist <- rexp(1000, rate = 1)
hist(exp_dist, probability = TRUE, xlab = NULL,
     main = "Histogram of exponential distribution", col = "turquoise")
lines(density(exp_dist), col = "red", lwd = 2, lty = 2)
abline(v = mean(exp_dist), col = "blue", lty = 2, lwd = 2)
```

Poisson distribution
```{r fig.height=4, fig.width=6}
set.seed(13092024)
pois_dist <- rpois(1000, lambda = 2.5)
hist(pois_dist, probability = TRUE, main = "Histogram of Poisson distribution",
     col = "turquoise", xlab = NULL)
lines(density(pois_dist), col = "red", lwd = 2, lty = 2)
abline(v = mean(pois_dist), col = "blue", lty = 2, lwd = 2)
```

07 Flow Control: Conditionals & Loops

if/else, for, while, and repeat, worked through a student-grading system, a dice-streak game, and looping over a folder of files to import them all at once.

▼ View R Markdown

○○○ r-programming/chapters/07-flow-control.Rmd

````
---
title: "07 - Flow Control: Conditionals & Loops"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

# `if` / `else`

```
if (condition) {
  ## code to run
}
```

```{r}
x <- 3
if (x < 4) {
  print(TRUE)
} else {
  print(FALSE)
}

# one-liner form
if (x < 4) print(TRUE) else print(FALSE)

# vectorized form: ifelse() works element-by-element on a whole vector
x <- c(3, 4, 6, 7, 0, 4)
ifelse(x < 4, TRUE, FALSE)
```

## Nested conditions: grading example

```{r}
set.seed(123)
marks <- round(rnorm(50, mean = 60, sd = 15), 2)
```

Categorize each mark: Fail [0-60), Pass [60-70), Good Pass [70-80),
Very Good Pass [80-85), Distinction [85+].

**Approach 1 — iterate over values directly:**
````

```{r}
category <- character(0)
for (m in marks) {
  if (m >= 0 & m < 60) {
    category <- c(category, "Fail")
  } else if (m >= 60 & m < 70) {
    category <- c(category, "Pass")
  } else if (m >= 70 & m < 80) {
    category <- c(category, "Good Pass")
  } else if (m >= 80 & m < 85) {
    category <- c(category, "Very Good Pass")
  } else {
    category <- c(category, "Distinction")
  }
}
head(df_marks <- data.frame(marks, category), 5)
```

Approach 2 – iterate over indices:
```{r}
category2 <- character()
for (i in seq_along(marks)) {
  if (marks[i] >= 0 & marks[i] < 60) {
    category2[i] <- "F"
  } else if (marks[i] >= 60 & marks[i] < 70) {
    category2[i] <- "P"
  } else if (marks[i] >= 70 & marks[i] < 80) {
    category2[i] <- "GP"
  } else if (marks[i] >= 80 & marks[i] < 85) {
    category2[i] <- "VGP"
  } else {
    category2[i] <- "D"
  }
}
df_marks$category2 <- category2
head(df_marks, 5)
```

Approach 3 – vectorized with nested `ifelse()`:
```{r}
category3 <- ifelse(marks < 60, "Fail",
              ifelse(marks < 70, "Pass",
               ifelse(marks < 80, "Good Pass",
                ifelse(marks < 85, "Very Good Pass", "Distinction"))))
all(df_marks$category == category3)
```

`for` Loops

```{r}
m <- 10
for (i in 1:m) print(i)
```

```{r}
for (i in 1:10) {
  if (i %% 3 == 0) {
    cat(i, "is a multiple of 3\n")
  } else if (i %% 2 == 0) {
    cat(i, "is a multiple of 2\n")
  } else {
    cat(i, "is neither a multiple of 2 nor 3\n")
  }
}
```

```

## Exercise 1: odd or even

```{r}
for (i in 1:10) {
  if (i %% 2 == 0) {
    cat(i, "is even\n")
  } else {
    cat(i, "is odd\n")
  }
}
```

Exercise 2: importing every CSV in a folder

Build the file path from parts using `paste()`/`paste0()`:
```{r}
paste("data", "csv", sep = "/")
paste0("data", "/", "csv")
```

List every CSV in a folder, then load them all into one list:
```{r}
dir.create("demo_csv", showWarnings = FALSE)
write.csv(head(iris, 3), "demo_csv/iris_part.csv", row.names = FALSE)
write.csv(head(mtcars, 3), "demo_csv/mtcars_part.csv", row.names = FALSE)

file_names <- dir("demo_csv", pattern = "\\.csv$", full.names = TRUE)
file_names

data_list <- list()
for (f in file_names) {
  data_list[[basename(f)]] <- read.csv(f)
}
str(data_list)
unlink("demo_csv", recursive = TRUE) # clean up the demo files
```

`while` Loops

```
while (condition) {
  ## code to run
  ## something that eventually makes condition FALSE
}
```

```{r}
i <- 0
while (i <= 10) {
  print(i * 2)
  i <- i + 10
}
```

Exercise: dice streak game

Print "YOU WON!" the moment three 5s are rolled in a row.
```{r}
set.seed(42)
streak <- 0
rolls <- 0
repeat {

````
  rolls <- rolls + 1
  roll <- sample(1:6, 1)
  streak <- if (roll == 5) streak + 1 else 0
  if (streak == 3) {
    cat("YOU WON! (after", rolls, "rolls)\n")
    break
  }
}
```

Exercise: interactive multiplication table

This one needs a live console, since `readline()` only works in an
interactive R session (not when a script is run non-interactively):
```{r eval=FALSE}
repeat {
  number <- as.numeric(readline(prompt = "Enter a number 0-9: "))
  cat(number, "x 2 table:\n")
  for (i in 1:10) cat(number, "x", i, "=", number * i, "\n")

  again <- readline(prompt = "Continue? (y/n): ")
  if (tolower(again) != "y") break
}
```

`repeat`

```
i <- 0
repeat {
  ## code to run
  i <- i + 1
  if (something_happens) break
}
```

```{r}
i <- 0
repeat {
  print(i)
  i <- i + 1
  if (i > 5) break
}
```
````

---

## 08 The Apply Family of Functions

apply, sapply, lapply, tapply, and vapply as faster, more readable alternatives to writing an explicit loop every time.

**▼ View R Markdown**

○○○ r-programming/chapters/08-apply-functions.Rmd

````
---
title: "08 - The Apply Family of Functions"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
````
````

````
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

The `apply` family replaces explicit loops when you want to run the same
function across rows, columns, or elements of a data structure.

# `apply()`

Applies a function across the margins (rows or columns) of a matrix.
```{r}
A <- matrix(1:4, nrow = 2)
A

avg <- function(x) sum(x) / length(x)
avg(1:10)
```

```{r}
apply(iris[-5], MARGIN = 2, summary) # MARGIN = 2 means "column-wise"
```

# `sapply()`

Like `apply()`, but works over vectors/lists and simplifies the result to
a vector or matrix where possible.
```{r}
sapply(A, sum) # does not respect matrix structure — flattens first
```

```{r}
coef_disp <- function(x) sd(x) / mean(x)
get_range  <- function(x) diff(range(x))

sapply(Filter(is.numeric, iris), get_range)

sapply(Filter(is.numeric, iris), function(x) c(
  min = min(x), max = max(x), std_dev = sd(x),
  Q1 = quantile(x, 0.25), Med = quantile(x, 0.5), Q3 = quantile(x, 0.75),
  disp = coef_disp(x)
))
```

`sapply()` falls back to returning a list when results aren't the same
length:
```{r}
sapply(iris, summary)
```

```{r}
sapply(1:10, function(x) x^2)
```

# `lapply()`

Always returns a list, one element per input element:
```{r}
a <- lapply(iris[-5], mean)
````
````

````
unlist(a)
```

# `tapply()`

Applies a function to subgroups of a vector, split by a factor:
```{r}
tapply(iris$Sepal.Length, iris$Species, mean)
```

# `vapply()`

Like `sapply()`, but requires you to specify the expected output shape —
safer for use inside functions/packages since it fails loudly on a mismatch:
```{r}
vapply(as.list(iris[-5]), quantile,
       FUN.VALUE = c("0%" = 0, "25%" = 0, "50%" = 0, "75%" = 0, "100%" = 0))
```
````

---

09 Writing Functions in R

Defining your own functions from scratch, then applying that to two worked exercises: replacing missing values with a chosen statistic, and solving a quadratic equation based on its discriminant.

▼ View R Markdown

○○○ r-programming/chapters/09-writing-functions.Rmd

````
---
title: "09 - Writing Functions in R"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

# Function Syntax

```
function_name <- function(arg1, arg2, ...) {
  ## code to run
}
```

```{r}
toss_coin <- function() {
  sample(6, size = 1)
}

set.seed(1)
faces <- sapply(1:10, function(i) toss_coin())
faces
````

```

```{r}
pp <- function(x) return(x + 1)
i <- 1
(i <- pp(i))
```

Exercises

1. Replace missing values with a summary statistic

```{r}
replace_missing <- function(x, fun = mean) {
  x[is.na(x)] <- fun(x, na.rm = TRUE)
  x
}

test_vec <- c(1, 2, NA, 4, NA, 6)
replace_missing(test_vec)
replace_missing(test_vec, fun = median)
```

2. Solve a quadratic equation and comment on its discriminant

For $ax^2 + bx + c = 0$:

```
                  ┌─ discriminant > 0 → two distinct real roots
compute disc = b² - 4ac ─┼─ discriminant = 0 → one repeated real root
                  └─ discriminant < 0 → two complex conjugate roots
```

```{r}
solve_quadratic <- function(a, b, c) {
  if (a == 0) stop("a must be non-zero for a quadratic equation")

  disc <- b^2 - 4 * a * c

  if (disc > 0) {
    message("Two distinct real roots")
    roots <- c((-b + sqrt(disc)) / (2 * a), (-b - sqrt(disc)) / (2 * a))
  } else if (disc == 0) {
    message("One repeated real root")
    roots <- -b / (2 * a)
  } else {
    message("Two complex conjugate roots")
    real_part <- -b / (2 * a)
    imag_part <- sqrt(-disc) / (2 * a)
    roots <- c(complex(real = real_part, imaginary = imag_part),
               complex(real = real_part, imaginary = -imag_part))
  }
  roots
}

solve_quadratic(1, -3, 2)  # disc > 0
solve_quadratic(1, 2, 1)   # disc == 0
solve_quadratic(1, 0, 1)   # disc < 0
```

---

10 **Packages**
```

Installing packages from CRAN, loading them with library() or pacman, reading their documentation, and the core dplyr verbs — select, filter, arrange, rename, mutate, summarise — for everyday data manipulation.

▼ View R Markdown

○○○ r-programming/chapters/10-packages.Rmd

````
---
title: "10 - Packages"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

A package is a documented bundle of functions, compiled code, and data.
Packages exist to make one specific job easy.

# Installing a Package

## From CRAN

Check the [full list of CRAN packages](https://cran.r-project.org/web/packages/available_packages_by_name.html).
If a package isn't already installed, install it with `install.packages()`.

```{r}
head(data.frame(installed.packages())[c(1, 3:5)], 5)
```

The installed-packages matrix has a `Package` column, easy to check with
`installed.packages()[, "Package"]`:
```{r}
all_packages <- installed.packages()[, "Package"]
"pacman" %in% all_packages
```

Install only if missing:
```{r message=FALSE, warning=FALSE}
if (!"pacman" %in% all_packages) {
  install.packages("pacman", repos = "http://cran.us.r-project.org")
}
```

# Loading a Package

`library()` (from base R) makes a package's functions available:
```{r warning=FALSE, message=FALSE}
library(pacman)
```

`pacman::p_load()` loads a whole list of packages at once, installing any
that are missing:
```{r eval=FALSE}
````
````

```
pkg_list <- c("tidyverse", "ggplot2", "lubridate", "flextable")
p_load(pkg_list, character.only = TRUE)
```

Package documentation

```{r eval=FALSE}
help(package = "pacman")
```
The full documentation is also available under RStudio's
Files | Plots | Packages | Help pane.

Suppressing package startup noise

`tidyverse` (and others) print a message on load that you usually don't
want in a rendered report:
```{r warning=FALSE, message=FALSE}
library(tidyverse)
```

The `dplyr` Package

`dplyr` is part of `tidyverse` and provides a consistent grammar for data
manipulation, which reads especially well combined with the pipe (`%>%`,
covered in Chapter 12).

The most-used `dplyr` verbs are:

* `select()` – keep a subset of columns
```{r}
iris %>% select(1:4) %>% names()
iris %>% select(-Species) %>% names()
```

* `filter()` – keep a subset of rows matching a condition
```{r}
iris %>% filter(Species == "setosa", Petal.Length > 1.4) %>% head(3)
```

* `arrange()` – reorder rows
```{r}
iris %>% arrange(desc(Sepal.Length)) %>% head(3)
```

* `rename()` – rename columns
```{r}
iris %>% rename(PetalLength = Petal.Length) %>% names()
```

* `mutate()` – add or transform columns
```{r}
iris %>% mutate(Ratio = Petal.Length / Petal.Width) %>% head(3)
```

* `summarise()` – collapse a group into one row
```{r}
iris %>% group_by(Species) %>% summarise(mean = mean(Petal.Width), .groups = "drop")
```

---

11 **Importing Data in R**

Reading built-in datasets, CSVs, and Excel files into R with base R, readr, and rio, then looping over a whole folder of files to load them all at once.

▼ View R Markdown

○○○ r-programming/chapters/11-importing-data.Rmd

````
---
title: "11 - Importing Data in R"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

> **Note on file paths:** the original course material referenced files
> under `./data/`. Those files aren't part of this repo, so every example
> below either uses a built-in dataset or generates a small temp file on
> the fly — copy the pattern and point it at your own files.

# Built-in Data

R ships with sample datasets ready to use, no import needed.
```{r}
data(iris)
str(iris)
summary(iris)
ncol(iris)
```

Checking the class of every column, timed with base R (or use the `tictoc`
package's `tic()`/`toc()` for nicer output if you have it installed):
```{r}
dtypes <- c()
start <- Sys.time()
for (i in 1:ncol(iris)) {
  dtypes[i] <- class(iris[[i]])
}
print(Sys.time() - start)
unique(dtypes)
```

Loading a dataset from a package without loading the whole package:
```{r}
data("spam", package = "kernlab")
str(spam[1:5])
```

# CSV Files

Three common ways to read a CSV, from least to most modern:

* `read.csv()` from base R
```{r}
tmp <- tempfile(fileext = ".csv")
````
````

```
write.csv(iris, tmp, row.names = FALSE)

dta <- read.csv(tmp)
head(dta, 3)
```
Notice `Species` comes back as `character` — pass `stringsAsFactors = TRUE`
to `read.csv()` if you want it as a factor instead.

* `read_csv()` from `readr` (loaded together with `tidyverse`)
```{r message=FALSE}
library(readr)
dta <- read_csv(tmp)
head(dta, 3)
```
Always check the messages/warnings `read_csv()` prints — they usually flag
real issues worth fixing.

* `import()` from `rio`, which auto-detects the file type
```{r message=FALSE}
library(rio)
head(import(tmp), 2)
unlink(tmp)
```

Excel files

`rio::import()` handles `.xlsx` the same way it handles CSV:
```{r message=FALSE}
tmp_xlsx <- tempfile(fileext = ".xlsx")
export(head(iris, 5), tmp_xlsx)
head(import(tmp_xlsx), 2)
unlink(tmp_xlsx)
```

Exercise: import every CSV in a folder with a loop

```{r}
dir.create("demo_csv", showWarnings = FALSE)
write.csv(head(iris, 3), "demo_csv/iris_part.csv", row.names = FALSE)
write.csv(head(mtcars, 3), "demo_csv/mtcars_part.csv", row.names = FALSE)

file_names <- dir("demo_csv", pattern = "\\.csv$")
file_names

data_list <- list()
for (f in file_names) {
  data_list[[f]] <- read.csv(file.path("demo_csv", f))
}
str(data_list)
unlink("demo_csv", recursive = TRUE)
```

---

12 The Pipe & Data Manipulation

Chaining steps left-to-right with %>% and the native |> pipe, reshaping data wide-to-long and back with reshape2/tidyr, and producing polished tables with flextable.

▼ View R Markdown

○○○ r-programming/chapters/12-pipe-and-dplyr.Rmd

````
---
title: "12 - The Pipe & Data Manipulation"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

# The Pipe

Two pipe operators are common in R:

* `%>%` from the `magrittr` package (loaded with `tidyverse`/`dplyr`)
* `|>` the base-R native pipe

RStudio shortcut: **Ctrl+Shift+M**.

Why bother? Because `f(g(h(x)))` reads inside-out, while
`x %>% h() %>% g() %>% f()` reads left to right, in the order things
actually happen.

```{r message=FALSE}
library(dplyr)
library(ggplot2)

iris %>% group_by(Species) %>%
  summarise(mean = mean(Petal.Width), .groups = "drop")

Gender <- factor(c("Female", "Male", "Male", "Female", "Female", "Male"))

Gender %>% table() %>% data.frame() %>%
  rename("Gender" = 1, Count = Freq) %>%
  mutate(Rel_freq = round(Count / sum(Count), 2)) %>%
  ggplot(aes(x = Gender, y = Rel_freq)) + geom_col()
```

Without the pipe, the same computation nests inside-out:
```{r}
summarise(group_by(iris, Species), mean = mean(Petal.Width))
```

# Data Manipulation with `tidyverse`

Loading `tidyverse` pulls in several packages at once: `ggplot2`, `tibble`,
`tidyr`, `readr`, `purrr`, `dplyr`, `stringr`, `forcats`, and `lubridate`.
There's no need to load each individually afterward.

## `dplyr`

Grammar-based data manipulation — see Chapter 10 for the core verbs
(`select`, `filter`, `arrange`, `rename`, `mutate`, `summarise`).

## `tibble`

A tibble is a modernized data frame: cleaner printing, no silent partial
matching, and no automatic factor conversion.
````

```{r}
library(tibble)
tb <- as_tibble(iris)
tb
glimpse(tb)
```

`reshape2`

Reshaping data between wide and long formats:
```{r message=FALSE, fig.height=3, fig.width=8}
library(reshape2)
library(tidyr)

view1 <- iris %>% group_by(Species) %>%
  summarise_if(is.numeric, list(mean = mean, sd = sd)) %>%
  melt(id.vars = "Species") %>%
  separate(variable, into = c("variable", "metric"), sep = "_")

view1
```

Pivoting back to wide format with `tidyr::pivot_wider()`:
```{r}
view2 <- view1 %>%
  pivot_wider(names_from = metric, values_from = value) %>%
  mutate(std_error = sd / sqrt(nrow(iris)))
view2
```

```{r fig.height=4, fig.width=8}
view2 %>% ggplot() +
  geom_bar(aes(x = Species, y = mean), stat = "identity", fill = "forestgreen", alpha = 0.5) +
  geom_errorbar(aes(x = Species, ymin = mean - std_error, ymax = mean + std_error),
                width = 0.4, colour = "orange", alpha = 0.9, linewidth = 0.5) +
  facet_wrap(~variable, ncol = 4)
```

Presenting Tables

`kableExtra`
Great for styling `knitr::kable()` tables in HTML/PDF reports — see the
[kableExtra cookbook chapter](https://bookdown.org/yihui/rmarkdown-cookbook/kableextra.html).

`flextable`
Produces polished tables that also render well in Word documents:
```{r message=FALSE, warning=FALSE}
library(broom)
library(flextable)

set.seed(123)
x <- rnorm(100, mean = 5, sd = 1.5)
z <- 4 + 8 * x + rnorm(100)
LR <- lm(z ~ x)

tidy(LR) %>% select(-statistic) %>% flextable() %>% autofit() %>% theme_box()
```

See the [Epi R Handbook tables chapter](https://epirhandbook.com/en/new_pages/tables_presentation.html)
for more `flextable` recipes.

`officer`
Generates and edits Word/PowerPoint files directly from R — see the
[officeverse guide](https://ardata-fr.github.io/officeverse/) for details.

13 Visualization: ggplot2 & plotly

Building layered grammar-of-graphics plots with ggplot2, then making them interactive with plotly and publication-ready with ggpubr.

▼ View R Markdown

○○○ r-programming/chapters/13-visualization.Rmd

````
---
title: "13 - Visualization: ggplot2 & plotly"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

# `ggplot2`

`ggplot2` builds plots in layers, following the "grammar of graphics."
```{r message=FALSE, fig.height=3, fig.width=8}
library(ggplot2)
library(dplyr)

iris %>%
  ggplot(aes(y = Petal.Length, fill = Species)) +
  geom_boxplot() +
  facet_grid(~Species) +
  theme_minimal()
```

For a full systematic introduction to `ggplot2`, see the
[Data Visualization chapters of R for Data Science](https://r4ds.hadley.nz/data-visualize.html)
or [this ggplot2 bookdown guide](https://bookdown.org/ozancanozdemir/introduction-to-ggplot2/).

# `plotly`

`plotly` turns static plots into interactive, hover-able ones:
```{r message=FALSE, fig.height=4, fig.width=6}
library(plotly)

plot_ly(data = iris %>% count(Species),
        labels = ~Species, values = ~n, type = "pie")
```

`ggpubr::ggpie()` gives a similar publication-ready pie chart built on
`ggplot2`:
```{r message=FALSE, fig.height=4, fig.width=6}
library(ggpubr)

ggpie(data = iris %>% count(Species) %>%
        mutate(label = paste0(Species, " (", round(100 * n / sum(n), 2), "%)")),
````
````

```
      x = "n", label = "label", color = "white",
      fill = "Species", lab.pos = "in")
```

See the [plotly R docs](https://plotly.com/r/) and
[this intro-to-DS plotting chapter](https://bookdown.org/ronsarafian/IntrotoDS/plotting.html)
for more.

---

14 Regular Expressions

Detecting, extracting, and replacing text patterns with base R's grepl/grep/sub/gsub and the more consistent stringr equivalents.

▼ View R Markdown

○○○ r-programming/chapters/14-regular-expressions.Rmd

````
---
title: "14 - Regular Expressions"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

A regular expression (regex) is a pattern used to match, extract, or
replace text. R's base functions for this all take a pattern as their
first serious argument and a character vector to search as the second.

```{r}
text <- c("dave@example.com", "call 555-1234", "no match here", "ID: R2024-001")
```

# Detecting Matches

`grepl()` returns `TRUE`/`FALSE` for each element:
```{r}
grepl("[0-9]+", text)
```

`grep()` returns matching positions, or the matching strings themselves
with `value = TRUE`:
```{r}
grep("[0-9]+", text)
grep("[0-9]+", text, value = TRUE)
```

# Replacing Matches

`sub()` replaces only the first match per string; `gsub()` replaces all:
```{r}
sub("[0-9]+", "#", text)
````
````

````
gsub("[0-9]+", "#", text)
```

# Extracting Matches

`regexpr()` finds the *first* match's position; `regmatches()` pulls out
the actual matched text:
```{r}
m <- regexpr("[0-9]+", text)
regmatches(text, m)
```

`gregexpr()` + `regmatches()` extracts *every* match per string:
```{r}
mg <- gregexpr("[0-9]+", text)
regmatches(text, mg)
```

# A Practical Pattern: Validating an Email

```{r}
grepl("^[A-Za-z]+@[A-Za-z]+\\.[a-z]+$", text)
```

# Common Building Blocks

| Pattern | Matches |
|---|---|
| `[0-9]` or `\\d` | any digit |
| `[a-zA-Z]` | any letter |
| `.` | any single character |
| `+` | one or more of the previous token |
| `*` | zero or more of the previous token |
| `^` / `$` | start / end of the string |
| `\\s` | whitespace |
| `\|` | alternation ("or") |

# `stringr` Equivalents

The `stringr` package (part of `tidyverse`) wraps the same regex engine in
more consistent, pipe-friendly functions:
```{r message=FALSE}
library(stringr)
str_detect(text, "[0-9]+")
str_extract(text, "[0-9]+")
str_extract_all(text, "[0-9]+")
str_replace_all(text, "[0-9]+", "#")
```

For deeper coverage, see
[DataCamp's regex in R tutorial](https://www.datacamp.com/tutorial/regex-r-regular-expressions-guide)
or [this regex tutorial](https://jfjelstul.github.io/regular-expressions-tutorial/).
````

---

15 Intro to Modeling: PCA, Clustering, KNN & Regression

A first tour of unsupervised and supervised learning: PCA and clustering to find structure with no labels, then KNN and logistic regression to actually predict a category.

▼ View R Markdown

○○○ r-programming/chapters/15-intro-to-modeling.Rmd
````

````
---
title: "15 - Intro to Modeling: PCA, Clustering, KNN & Regression"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

A first tour of the modeling techniques R is most commonly used for,
building on the eigen-decomposition and linear model work from earlier
chapters.

# Principal Component Analysis

Chapter 3 walked through PCA by hand: center the data, eigen-decompose its
covariance matrix, and project onto the top eigenvectors. In practice,
`prcomp()` does all of that for you, plus proper scaling:
```{r}
data(iris)
pca <- prcomp(iris[, -5], center = TRUE, scale. = TRUE)
summary(pca)
```

`Proportion of Variance` shows how much information each component
captures — here the first two components already explain most of the
spread in the data.
```{r fig.height=4, fig.width=6}
plot(pca$x[, 1:2], col = iris$Species, pch = 16,
     xlab = "PC1", ylab = "PC2", main = "iris projected onto PC1/PC2")
legend("topright", legend = levels(iris$Species),
       col = 1:3, pch = 16, bty = "n")
```

# Clustering

## K-means

Partitions data into `k` groups by minimizing within-cluster distance.
Being unsupervised, it never sees `Species` — comparing its output to the
real labels afterward is just for evaluation.
```{r}
set.seed(1)
km <- kmeans(iris[, -5], centers = 3, nstart = 10)
table(cluster = km$cluster, species = iris$Species)
```

## Hierarchical clustering

Builds a tree of nested clusters instead of a single flat partition:
```{r fig.height=4, fig.width=7}
d <- dist(iris[1:20, -5]) # a small subset, so the dendrogram stays readable
hc <- hclust(d, method = "complete")
plot(hc, main = "Hierarchical clustering (first 20 flowers)")

cutree(hc, k = 3) # cut the tree into 3 groups
````
````

```
```

K-Nearest Neighbor (KNN)

A supervised classifier: label a new point by majority vote among its `k`
closest neighbors in the training data.
```{r message=FALSE}
library(class)
set.seed(1)
idx <- sample(1:nrow(iris), 0.7 * nrow(iris))
train <- iris[idx, ]
test  <- iris[-idx, ]

pred <- knn(train[, -5], test[, -5], cl = train$Species, k = 5)
mean(pred == test$Species) # accuracy on the held-out test set
table(predicted = pred, actual = test$Species)
```

Simple Linear Regression

Already covered in Chapter 6's simulated-data example
(`z ~ x`, fitted with `lm()`). A quick recap on `iris`:
```{r}
lr <- lm(Petal.Length ~ Petal.Width, data = iris)
coef(lr)
```
Each additional cm of petal width is associated with roughly
`r round(coef(lr)[[2]], 2)` cm more petal length.

Logistic Regression

For a categorical (here, binary) outcome, swap `lm()` for `glm()` with
`family = binomial`. Predicting *virginica* vs. *versicolor* from sepal
measurements:
```{r}
sub <- subset(iris, Species %in% c("versicolor", "virginica"))
sub$is_virginica <- factor(ifelse(sub$Species == "virginica", 1, 0))

logit <- glm(is_virginica ~ Sepal.Length + Sepal.Width, data = sub, family = binomial)
summary(logit)$coefficients
```

Turning predicted probabilities into class predictions at a 0.5 threshold:
```{r}
pred_prob  <- predict(logit, type = "response")
pred_class <- ifelse(pred_prob > 0.5, 1, 0)

mean(pred_class == sub$is_virginica) # accuracy
table(predicted = pred_class, actual = sub$is_virginica)
```
```

---

## 16 LaTeX in R Markdown

Writing inline and display math alongside your R code — a natural bridge into Part 3, where LaTeX becomes the main event.

**▼ View R Markdown**

○○○ r-programming/chapters/16-latex-in-rmarkdown.Rmd
```

````
---
title: "16 - LaTeX in R Markdown"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

R Markdown (and Quarto) render LaTeX math directly, both inline and as
display equations — useful for writing up the statistics behind a model
alongside the code that fits it.

# Inline Math

Wrapping an expression in single dollar signs keeps it inline with text:
the variance of a real-valued variable $X = (X_1, \ldots, X_n)$ is written
right in the sentence like this.

Source:
```
The variance of a real-valued variable $X = (X_1, \ldots, X_n)$ is given by:
```

# Display Math

Double dollar signs (or a `$$...$$` block) center the equation on its own
line:

$$
\text{Var}(X) =
\left[\frac{1}{n}\sum_{i=1}^n\left(X_i-\frac{1}{n}
\sum_{i=1}^nX_i\right)^2\right]^\frac{1}{2}
$$

Source:
```
$$
\text{Var}(X) =
\left[\frac{1}{n}\sum_{i=1}^n\left(X_i-\frac{1}{n}
\sum_{i=1}^nX_i\right)^2\right]^\frac{1}{2}
$$
```

# Mixing LaTeX and R

Math and code can sit side by side in the same document — describe a
function mathematically, then define and run it:

$$f(x) = x^3$$

```{r}
cube <- function(x) x^3
cube(3)
```
````

```
This is the same LaTeX engine used to render PDF output (via `tinytex`,
set up in Chapter 0), so anything that displays correctly here will also
render correctly in a knitted PDF.
```

03

LaTeX

22 sections on typesetting documents by writing, not clicking.

Part 3 of 3

LaTeX

Typesetting documents by writing, not clicking. 22 slides from a blank file to your first properly structured document.

Overleaf's full tutorialTry it online (Overleaf)

Use → / click Next to begin

Concepts

What Is LaTeX?

Most word processors are **WYSIWYG** — what you see is what you get. You click, drag, and format a page directly, and the tool shows you the result as you go.

LaTeX works the opposite way. You write a plain text file full of commands that describe *what things are* — a section, a table, an equation — not how they should look pixel-by-pixel. A program called a **TeX engine** then reads that file and compiles it into a finished PDF.

The upshot: you focus on content and structure, and LaTeX handles the visual formatting for you, consistently, every time.

Concepts

Why Learn LaTeX?

- Handles complex math, technical notation, and large tables far more gracefully than a word processor.
- Numbers figures, equations, citations, and table-of-contents entries automatically — renumber freely without breaking anything.
- Separates content from style: rewrite what a document looks like without touching a word of what it says.
- Extensible through thousands of free, community-written packages.

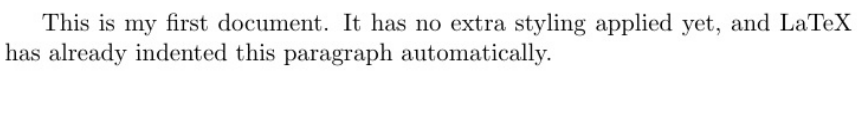
It's not the right tool for everything — free-form magazine-style layouts are still easier in a visual design tool. But for anything technical, academic, or reference-heavy, it earns its learning curve fast.

Your First Document

Documentclass & Body

main.tex

```
\documentclass{article}
\begin{document}
This is my first document. It has no extra
styling applied yet, and LaTeX has already
indented this paragraph automatically.
\end{document}
```



rendered PDF

`\documentclass` sets the document type. Everything between `\begin{document}` and `\end{document}` is the body — the part that actually gets typeset.

Structure

The Preamble

Everything *before* `\begin{document}` is called the **preamble** — the document's setup section.

preamble

```
\documentclass[12pt, a4paper]{article}
\usepackage{amsmath}
```

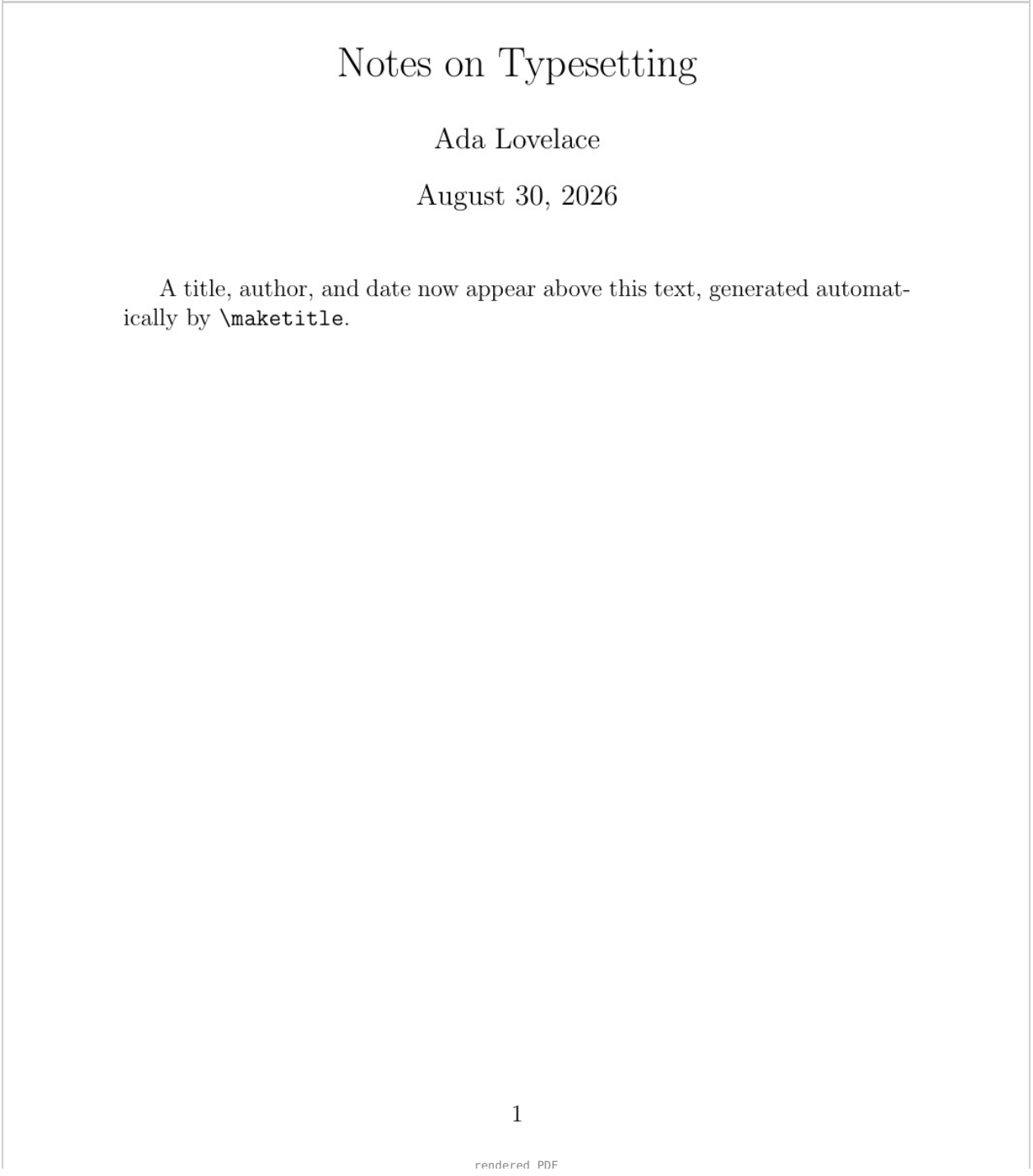
- Options inside `[...]` configure the class — here, 12-point type on A4 paper.
- `\usepackage{...}` loads extra functionality — more on packages in a few slides.

Structure

Title, Author & Date

main.tex

```
\documentclass[12pt, letterpaper]{article}
\title{Notes on Typesetting}
\author{Ada Lovelace}
\date{\today}
\begin{document}
\maketitle
A title, author, and date now appear above
this text, generated automatically by
\maketitle.
\end{document}
```



rendered PDF

rendered PDF

`\date{\today}` inserts the actual compile date automatically, every single time you rebuild the document.

Structure

Comments

main.tex

```
% Everything after a percent sign is ignored.
% Use comments for to-do notes, explanations,
% or to disable a line while debugging.
\documentclass{article}
```

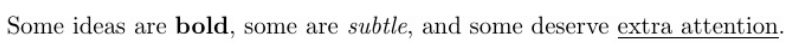
A `%` at the start of a line — or anywhere mid-line — comments out everything after it. Nothing after a `%` gets typeset.

Formatting

Bold, Italic & Underline

main.tex

```
Some ideas are \textbf{bold}, some
are \textit{subtle}, and some
deserve \underline{extra attention}.
```



rendered PDF

There's also `\emph{...}`, which adapts to context: italic inside upright text, but it flips to upright if you're already inside an italicized block.

Formatting

Lists

main.tex

```
\begin{itemize}
  \item First point
  \item Second point
\end{itemize}

\begin{enumerate}
  \item Step one
  \item Step two
\end{enumerate}
```



rendered PDF

`itemize` gives bullets, `enumerate` gives automatic numbering — same `\item` syntax either way.

Media

Adding Images

Importing a graphic needs the `graphicx` package, loaded in the preamble:

main.tex

```
\usepackage{graphicx}
\graphicspath{{figures/}}

\includegraphics[width=0.75\textwidth]{diagram}
```

- `\graphicspath` tells LaTeX which folder to look in.
- Give the filename without its extension — LaTeX searches supported formats for you.
- `width=0.75\textwidth` scales the image relative to the text column, not an absolute size.

Media

Captions, Labels & Cross-References

main.tex

```
\begin{figure}[h]
  \centering
  \includegraphics[width=0.6\textwidth]{diagram}
  \caption{A labeled diagram.}
  \label{fig:diagram}
\end{figure}

See Figure~\ref{fig:diagram} on
page~\pageref{fig:diagram}.
```

`\label` + `\ref` is the whole trick: LaTeX tracks the actual figure number for you. Reorder or insert figures freely — every reference updates itself on the next compile.

Math

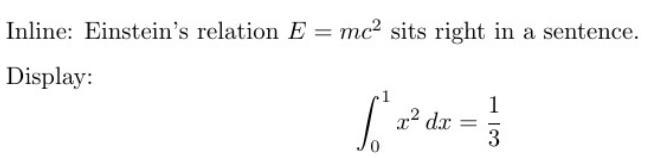
Math Mode: Inline vs. Display

main.tex

```
Inline: Einstein's relation
$E=mc^2$ sits right in a sentence.

Display:
\[ \int_0^1 x^2\,dx = \frac{1}{3} \]

Greek and subscripts:
$\alpha$, $\beta$, $\Omega$, $x_i^2$
```



rendered PDF

Single `$...$` keeps math inline with text. `\[...\]` centers an equation on its own line — prefer it over the old-style `$$...$$`, which modern LaTeX advises against.

Math

Common Math Building Blocks

Want	Write
Fraction	`\frac{a}{b}`
Subscript / superscript	`a_b` / `a^b`
Square root	`\sqrt{x}`
Integral	`\int_0^1 ... \, dx`
Summation	`\sum_{i=1}^n`
Greek letters	`\alpha \beta \Omega`
Named functions	`\sin \cos \log`

All of these work inside either `$...$` or `\[...\]`. Named functions like `\sin` matter — plain `sin(x)` renders italicized as if it were three separate variables multiplied together.

Document Structure

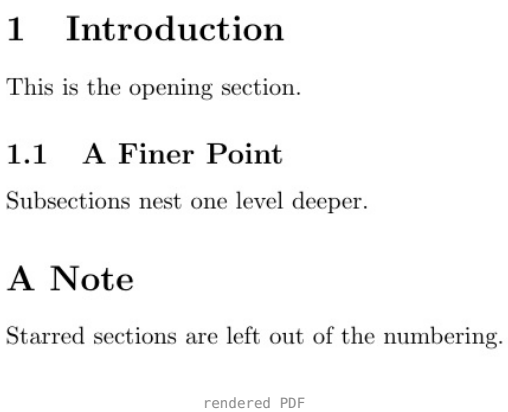
Sections & Chapters

main.tex

```
\section{Introduction}
This is the opening section.

\subsection{A Finer Point}
Subsections nest one level deeper.

\section*{A Note}
Starred sections skip numbering.
```



rendered PDF

`\chapter` and `\part` only exist in the `report` and `book` classes — `article` tops out at `\section`. Numbering is automatic unless you use the starred (`*`) form.

Document Structure

Paragraphs & Line Breaks

main.tex

```
First paragraph.

Second paragraph starts after a blank line.

Third paragraph continues on\\
a manual new line, same paragraph.
```

- A blank line starts a new paragraph — LaTeX indents it automatically.
- `\\` (or `\newline`) breaks the line without starting a new paragraph.

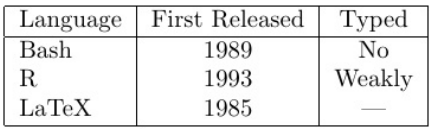
Avoid stacking `\\\\` to fake extra spacing between paragraphs — it fights LaTeX's own spacing rules. Use the `parskip` package instead if you want visible gaps.

Tables

Building a Table

main.tex

```
\begin{tabular}{|l|c|c|}
\hline
Language & First Released & Typed \\
\hline
Bash & 1989 & No \\
R & 1993 & Weakly \\
LaTeX & 1985 & --- \\
\hline
\end{tabular}
```



Language	First Released	Typed
Bash	1989	No
R	1993	Weakly
LaTeX	1985	—

rendered PDF

`{|l|c|c|}` declares three columns (left/center/center-aligned) with vertical rules between them. `&` separates cells, `\\` ends a row, and `\hline` draws a horizontal rule.

Document Structure

Table of Contents

main.tex

```
\tableofcontents

\section{Background}
...
\section{Findings}
...
```

One command builds and numbers the entire table of contents from whatever `\section` / `\subsection` commands already exist in your document. Compile twice — the first pass collects the section numbers and page numbers, the second pass writes them into the contents page.

Packages

Loading Packages & Options

main.tex

```
\usepackage{geometry}
\usepackage[margin=1in]{geometry}
```

Options inside `[...]` configure a package's behavior — here, setting every page margin to 1 inch. Without options, a package just loads with its defaults.

If LaTeX can't find the package's underlying `.sty` file, it stops with a clear "file not found" error rather than silently failing.

Packages

Finding Packages: CTAN & TeX Live

CTAN (the Comprehensive TeX Archive Network) hosts thousands of free, community-contributed packages covering everything from chemistry notation to chess diagrams.

TeX Live bundles a large, tested snapshot of CTAN into one installable distribution, refreshed annually. Overleaf, and most desktop LaTeX installs, run on TeX Live — so a package can exist on CTAN without yet being available to you if it's very new or has licensing/platform quirks.

Browse CTANAbout TeX Live

Reference

Cheat Sheet

Command	Does
`\documentclass{article}`	Sets the document type
`\usepackage{name}`	Loads extra functionality
`\section{...}`	Numbered heading
`\textbf{} \textit{}`	Bold / italic
`itemize` / `enumerate`	Bulleted / numbered list
`$...$` / `\[...\]`	Inline / display math
`tabular`	Table environment
`\label` / `\ref`	Cross-reference anything
`\tableofcontents`	Auto-generated contents page

Keep Going

Where to Go Next

This deck covers the same ground as Overleaf's own free introduction, condensed and restructured as slides — their version has more detail on every topic and is worth reading end to end.

Overleaf: Learn LaTeX in 30 minutesOverleaf documentation home

Beyond the basics: bibliography management (`biblatex` / `bibtex`), the `Beamer` class for building actual slide presentations in LaTeX itself, and `TikZ` for drawing diagrams directly in your document.

Series Complete

Bash → R → LaTeX

All three parts of the course are now live.